Beyond the Syllabus

Greedy Correctness, Matroids, and Online Algorithms with Predictions

Week 4 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This week you proved earliest-finish-time-first optimal for interval scheduling using an **exchange argument**, and saw the same proof style reused for interval partitioning, minimizing lateness, and job sequencing.
- Three questions a curious student should ask next:
  1. Exchange arguments feel ad hoc, tailored to each problem. Is there a single **structural condition** on the underlying combinatorial problem that tells you, in advance, whether *any* greedy rule can possibly be optimal?
  2. Every algorithm this week assumed you see the *whole* instance before sorting. What happens when requests arrive **online**, one at a time, and you must accept or reject immediately?
  3. If you had a (possibly imperfect) **prediction** about future requests – say, from a machine-learning forecast – could you beat the classical online lower bounds?

How this supplement is different from a survey

Every citation below was checked against its actual paper/venue before this slide was written. Where a specific bound or year is quoted, it is sourced to a specific, verifiable paper.

When does *any* greedy rule work?

Rado–Edmonds theorem (classical)

Let $(E, \mathcal{I})$ be an independence system (a ground set E with a family $\mathcal{I}$ of "independent" subsets, downward closed). For **every** choice of nonnegative weights, the greedy algorithm (sort by weight, add an element if it keeps the current set independent) finds a maximum-weight independent set **if and only if** $(E, \mathcal{I})$ is a **matroid**: $\mathcal{I}$ is hereditary, and whenever $A, B \in \mathcal{I}$ with $|A| < |B|$, some $x \in B \setminus A$ has $A \cup \{x\} \in \mathcal{I}$.

- This is exactly why earliest-finish-time-first worked: the exchange step in this week's proof (swap a piece of O for a piece of G without losing feasibility) is a special case of the matroid exchange property.
- It also explains *why* some greedy rules fail: the "compatible intervals" independence system is close to, but not literally, a matroid in general – which is why the correctness proof needs the specific finish-time ordering, not an arbitrary one.

From matroids to submodular functions

- Many modern applications (sensor placement, feature selection, influence maximization) replace "maximize total weight" with maximize a **submodular** set function f: adding an element helps *less* as the set grows (diminishing returns), formally $f(A \cup \{x\}) - f(A) \geq f(B \cup \{x\}) - f(B)$ for $A \subseteq B$.
- For a single cardinality constraint ($|A| \leq k$), the plain greedy (repeatedly add the element with the largest marginal gain) is only a $(1 - 1/e)$-approximation, not exact – diminishing returns break the exchange argument's "no worse off" step.
- Under a general **matroid** constraint (not just $|A| \leq k$), the continuous-greedy method of Calınescu, Chekuri, Pál & Vondrák (2011) still achieves $(1 - 1/e)$, matching the best possible unless $P = NP$.

A very recent refinement: how many oracle calls does it take?

Kobayashi & Terao, ICALP 2024 – "Subquadratic Submodular Maximization with a General Matroid Constraint"

Achieves a $(1 - 1/e - \varepsilon)$-approximation using only $\widetilde{O}_\varepsilon(\sqrt{r}\, n)$ independence-oracle and value-oracle queries, improving the previous best bound of $\widetilde{O}_\varepsilon(r^2 + \sqrt{r}\, n)$ due to Buchbinder, Feldman & Schwartz. (LIPIcs vol. 297, pp. 100:1–100:19; arXiv:2405.00359.)

- Here n = ground-set size, r = matroid rank. Concretely, at $n = 10^6, r = 10^5$: the old bound's r^2 term is 10^{10}, while the new bound's $\sqrt{r}\, n \approx 3.16 \times 10^8$ – a $\sim 30\times$ reduction in the number of expensive feasibility/value checks, for the *same* approximation guarantee.
- Same flavor as this week's lesson: the *quality* bound $(1 - 1/e)$ was already known; the 2024 advance is entirely about **how cheaply** you can reach it – exactly the "prove it correct, then make it fast" arc this course follows.

The online version of interval scheduling

- This week's algorithm assumes all n requests are known before you sort by finish time. In an **online** version, requests arrive one at a time and you must accept or reject *immediately*, with no take-backs.
- A short adversary argument shows no deterministic online algorithm can achieve a constant competitive ratio in general: the adversary can always follow an accepted long interval with many short, mutually compatible intervals that the long one made infeasible to accept, and can do this with unboundedly bad ratio. (This is a standard, easy folklore argument in online algorithms – unlike the dated results below, it needs no specific citation.)
- So classical worst-case online algorithms for this exact problem are hopeless in general. What changed recently is a framework for doing much better when you have **side information**.

The "algorithms with predictions" framework

Mitzenmacher & Vassilvitskii, *Commun. ACM* 65(7), 2022, pp. 33–35. DOI: 10.1145/3528087.

Augment a classical online algorithm with a (possibly wrong) machine-learned **prediction** about the future. Measure it on two axes:

- **Consistency**: the competitive ratio when the prediction is *perfect*.
- **Robustness**: the competitive ratio in the *worst case*, even if the prediction is completely wrong.

- The goal is a smooth trade-off: near-optimal (consistency close to 1) when the forecast is good, but never worse than a fixed worst-case bound (robustness) if it is bad – you never do worse than "ignore the prediction and fall back to a classical online algorithm."

Applied to exactly this week's problem

Boyar, Favrholdt, Kamali & Larsen – "Online Interval Scheduling with Predictions" (WADS 2023; journal version arXiv:2302.13701)

Studies online interval scheduling where the algorithm is given a (possibly erroneous) prediction of the *full request sequence*. They prove asymptotically **tight** upper and lower bounds on the achievable competitive ratio as a function of the prediction error, and tight consistency-vs-robustness trade-off curves – then validate the theory on real scheduling workloads.

- This is a direct descendant of today's lecture: same problem (maximize accepted non-overlapping intervals), same notion of "compatible," but with the single-shot assumption of the offline greedy replaced by an online, predicted setting.

As current as it gets: a Nov. 2025 follow-up

Antoniadis, Shahheidar, Shahkarami & Soltani – "A Switching Framework for Online Interval Scheduling with Predictions" (AAAI 2026; posted to arXiv Nov. 2025, arXiv:2511.16194)

Instead of committing to one predictor, maintain several candidate predictors simultaneously and **switch** between them adaptively as their track records diverge – hedging against betting everything on a single, possibly bad, forecast.

- This paper is still working its way through the AAAI 2026 publication pipeline as this deck is written – about as close to "the current edge of the field" as a course supplement can responsibly cite.
- The core idea – *don't trust one prediction, hedge across several, and adapt* – is a natural generalization of the exchange-argument mindset from today's lecture: keep whichever candidate is currently "winning," and be willing to swap.

Summary

1. **Matroids explain when greedy works at all**: the Rado–Edmonds theorem shows exchange-argument-style correctness is not a coincidence for interval scheduling – it is a consequence of an underlying matroid structure.
2. **Submodularity is the natural next step** once "maximize total weight" becomes "maximize with diminishing returns"; Kobayashi & Terao (ICALP 2024) show the classical $(1 - 1/e)$ quality bound can now be reached with far fewer oracle queries.
3. **Removing the offline assumption reopens the problem**: with no information about the future, online interval scheduling has no constant competitive ratio; with a prediction, Boyar et al. (2023) give tight consistency/robustness trade-offs, and Antoniadis et al. (AAAI 2026) push this to hedging across multiple predictors.
4. The throughline across all of it: **correctness first, then efficiency, then robustness to a changing model of the input** – the same arc this course follows week over week.

Looking ahead

Next week (Kleinberg & Tardos, Ch. 4 continued) proves Dijkstra's algorithm and Kruskal/Prim's MST algorithms optimal with the *same* two proof templates – and Union-Find from Week 3 reappears as Kruskal's core data structure.